



ULAB

UNIVERSITY OF LIBERAL ARTS
BANGLADESH

Project Report

Submitted to:

Md. Nazmul Abdal
Lecturer
Department of CSE
University of Liberal Arts Bangladesh

Submitted By:

Group Members' ID	Group Members' Name
243014003	Md. Basituzzaman
243014111	Mohammad Razin Masud

Course Title: Data Structures Lab

Course Code: CSE 1302

Section: 6

Semester: Summer 2025

Date of Submission: 21/08/2025

Title: Word Puzzle Game

Introduction:

This project is a simple word puzzle game written in C. The player is given a random set of letters and needs to form valid words from them. The program checks each word using a dictionary stored in a Trie data structure. The game rewards players with points based on word length and stores scores in a dynamic scoreboard. It is designed with multiple levels, each providing new challenges. This project demonstrates how data structures can be applied to create real, interactive applications.

Motivation:

- To apply theoretical knowledge of data structures into a working project.
- To develop a fun and educational game that helps improve vocabulary and logical thinking.
- To practice C programming skills, including file handling, dynamic memory, and algorithm design.

Objectives:

- To implement a Trie data structure for fast and efficient word searching.
- To design a multi-level puzzle game where players can progress through different challenges.
- To build a scoring system that rewards longer words and stores results dynamically.
- To use file handling for loading dictionary words, making the game realistic and extensible.
- To display a scoreboard that shows player name, level, and total points.

Background Theory:

This project uses basic C concepts :

Trie Data Structure : A Trie is a tree-like data structure that stores words where each node represents a letter. Searching for a word in a Trie is very fast because lookup time depends only on the length of the word, not on the total number of words. In this project, the dictionary is stored in a Trie so that player input words can be checked quickly and efficiently. This makes word validation faster compared to using arrays or lists.

Linked List :

A linked list is a sequence of nodes where each node holds data and a pointer to the next node. It is useful when the number of elements is not fixed. The scoreboard uses a linked list to store player names, levels, and scores. This allows dynamic addition of new scores without needing to predefine the size.

File Handling :

File handling allows programs to read from and write to external files. The dictionary words are loaded from a file (words.txt) instead of hardcoding them in the program. This makes the game flexible and easy to update with new words.

Randomization

Randomization rearranges or generates values unpredictably. The letters of the puzzle are shuffled randomly before being displayed to the player. This ensures that every playthrough feels different and prevents repetition.

Features and Functionalities:

- Multi-level gameplay with unique puzzles.
- Fast word validation using Trie.
- Score calculation based on word length.
- Dynamic scoreboard for storing player results.
- Dictionary file support for flexible word sets.
- Random letter shuffling for variation in gameplay.

Modern Tools:

IDE: Code::Blocks 20.03

Compiler: GCC 9.3.0

Operating System: Windows 10

Algorithm Design:

- 1.Start the program and take the player's name.
- 2.Create a Trie and load dictionary words from a file into it.
- 3.For each level:
 - Generate a set of puzzle letters from predefined words.
 - Shuffle the letters and display them to the player.
 - Take words as input until the player types "end".
 - For each word:
 - Check if it can be formed from the puzzle letters.
 - If yes, check if it exists in the Trie dictionary.
 - If valid, calculate score based on word length and add to total.
 - If not valid, display an error message.
 - Store the player's level score in the scoreboard (linked list).
 - Ask if the player wants to continue to the next level.
- 5.After finishing, display the final scoreboard with all results.
- 6.End the program.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>
#include <time.h>
```

```

#include <ctype.h>

#define ALPHABET_SIZE 26
#define MAX_WORD_LEN 20
#define PUZZLE_SIZE 7
#define MAX_LEVELS 3

typedef struct TrieNode {
    struct TrieNode *children[ALPHABET_SIZE];
    bool isEndOfWord;
} TrieNode;

TrieNode* createNode() {
    TrieNode *node = (TrieNode *)malloc(sizeof(TrieNode));
    node->isEndOfWord = false;
    for (int i = 0; i < ALPHABET_SIZE; i++)
        node->children[i] = NULL;
    return node;
}

void insert(TrieNode *root, const char *word) {
    TrieNode *current = root;
    while (*word) {
        int index = *word - 'a';
        if (!current->children[index])
            current->children[index] = createNode();
        current = current->children[index];
        word++;
    }
    current->isEndOfWord = true;
}

bool search(TrieNode *root, const char *word) {
    TrieNode *current = root;
    while (*word) {
        int index = *word - 'a';
        if (!current->children[index])
            return false;
        current = current->children[index];
        word++;
    }

```

```

    }
    return current && current->isEndOfWord;
}

```

```

typedef struct ScoreNode {
    char player[50];
    int level;
    int score;
    struct ScoreNode *next;
} ScoreNode;

```

```

ScoreNode* addScore(ScoreNode *head, const char *player, int level, int score) {
    ScoreNode *newNode = (ScoreNode *)malloc(sizeof(ScoreNode));
    strcpy(newNode->player, player);
    newNode->level = level;
    newNode->score = score;
    newNode->next = head;
    return newNode;
}

```

```

void showScoreboard(ScoreNode *head) {
    printf("\nScoreboard:\n");
    while (head) {
        printf("Player: %s | Level %d | %d points\n", head->player, head->level, head->score);
        head = head->next;
    }
}

```

```

void shuffle(char *letters, int size) {
    for (int i = size - 1; i > 0; i--) {
        int j = rand() % (i + 1);
        char temp = letters[i];
        letters[i] = letters[j];
        letters[j] = temp;
    }
}

```

```

const char levelWords[MAX_LEVELS][20][MAX_WORD_LEN] = {
    {"cat", "dog", "bat", "rat", "tab", "bag", "god", "cab", "tag", "bad"},
    {"tree", "bee", "see", "rest", "bet", "set", "tea", "eat", "ate", "net"},
}

```

```
    {"sun", "run", "fun", "fur", "rug", "gun", "gum", "bug", "cup", "cap"}  
};
```

```
void loadDictionary(const char *filename, TrieNode *root) {  
    FILE *fp = fopen(filename, "r");  
    if (!fp) {  
        return;  
    }  

```

```
    char buffer[MAX_WORD_LEN];  
    while (fgets(buffer, sizeof(buffer), fp)) {  
        buffer[strcspn(buffer, "\r\n")] = 0;
```

```
        if (strlen(buffer) == 0)  
            continue;
```

```
        for (int i = 0; buffer[i]; i++) {  
            buffer[i] = tolower((unsigned char)buffer[i]);  
        }
```

```
        int len = strlen(buffer);  
        if (len > PUZZLE_SIZE)  
            continue;
```

```
        bool valid = true;  
        for (int i = 0; i < len; i++) {  
            if (buffer[i] < 'a' || buffer[i] > 'z') {  
                valid = false;  
                break;  
            }  
        }  
        if (!valid)  
            continue;
```

```
        insert(root, buffer);  
    }  
    fclose(fp);  
}
```

```
bool canFormWord(const char *word, const char *puzzle, int puzzleLen) {
```

```

int letterCount[26] = {0};
int wordCount[26] = {0};

for (int i = 0; i < puzzleLen; i++)
    letterCount[puzzle[i] - 'a']++;

for (int i = 0; word[i]; i++) {
    char c = word[i];
    if (c < 'a' || c > 'z')
        return false;
    wordCount[c - 'a']++;
    if (wordCount[c - 'a'] > letterCount[c - 'a'])
        return false;
}
return true;
}

void flushInput() {
    int c;
    while ((c = getchar()) != '\n' && c != EOF);
}

int main() {
    srand(time(0));

    char currentPlayer[50];
    printf("Enter your player name: ");
    scanf("%49s", currentPlayer);
    flushInput();

    ScoreNode *scoreList = NULL;

    TrieNode *dictionary = createNode();
    loadDictionary("words.txt", dictionary);

    for (int levelIndex = 0; levelIndex < MAX_LEVELS; levelIndex++) {
        printf("\nStarting Level %d...\n", levelIndex + 1);

        char puzzle[PUZZLE_SIZE];
        int used[26] = {0}, pIndex = 0;

```



```

for (int i = 0; i < 10 && pIndex < PUZZLE_SIZE; i++) {
    const char *word = levelWords[levelIndex][i];
    for (int j = 0; word[j] && pIndex < PUZZLE_SIZE; j++) {
        int letter = word[j] - 'a';
        if (!used[letter]) {
            used[letter] = 1;
            puzzle[pIndex++] = word[j];
        }
    }
}

shuffle(puzzle, pIndex);

printf("\nUse these letters:\n\n");
for (int i = 0; i < pIndex; i++)
    printf("%c ", puzzle[i]);
printf("\n\nEnter words (type 'end' to finish this level):\n");

char input[MAX_WORD_LEN];
int levelScore = 0;

while (1) {
    printf("-> ");
    if (scanf("%19s", input) != 1) {
        break;
    }
    flushInput();

    for (int i = 0; input[i]; i++)
        input[i] = tolower((unsigned char)input[i]);

    if (strcmp(input, "end") == 0)
        break;

    if (!canFormWord(input, puzzle, pIndex)) {
        printf("%s is invalid: uses letters outside the puzzle.\n", input);
        continue;
    }
}

```

```

    if (search(dictionary, input)) {
        int score = strlen(input) * 10;
        levelScore += score;
        printf("Great job! You nailed '%s' (+%d points)\n", input, score);
    } else {
        printf("%s is not a valid dictionary word.\n", input);
    }
}

scoreList = addScore(scoreList, currentPlayer, levelIndex + 1, levelScore);
printf("\nLevel %d Complete! You scored: %d points\n", levelIndex + 1, levelScore);

if (levelIndex + 1 < MAX_LEVELS) {
    char choice[10];
    printf("Do you want to play the next level? (yes/no): ");
    if (scanf("%9s", choice) != 1) {
        break;
    }
    flushInput();
    for (int i = 0; choice[i]; i++)
        choice[i] = tolower((unsigned char)choice[i]);
    if (strcmp(choice, "no") == 0 || strcmp(choice, "n") == 0)
        break;
} else {
    printf("You've completed all levels!\n");
}
}

showScoreboard(scoreList);

return 0;
}

```

Outcome:

```
F:\ULAB\wordcrazemew.exe
Enter your player name: basit

Starting Level 1...

Use these letters:

b c a g d t o

Enter words (type 'end' to finish this level):
-> cat
Great job! You nailed 'cat' (+30 points)
-> dog
Great job! You nailed 'dog' (+30 points)
-> end

Level 1 Complete! You scored: 60 points

F:\ULAB\wordcrazemew.exe
Level 1 Complete! You scored: 60 points
Do you want to play the next level? (yes/no): yes

Starting Level 2...

Use these letters:

a t b e s r n

Enter words (type 'end' to finish this level):
-> best
Great job! You nailed 'best' (+40 points)
-> sent
Great job! You nailed 'sent' (+40 points)
-> nest
Great job! You nailed 'nest' (+40 points)
-> set
Great job! You nailed 'set' (+30 points)
-> end

Level 2 Complete! You scored: 150 points
```

```
F:\ULAB\wordcrazemew.exe

Level 3 Complete! You scored: 60 points
You've completed all levels!

Scoreboard:
Player: basit | Level 3 | 60 points
Player: basit | Level 2 | 150 points
Player: basit | Level 1 | 60 points

Process returned 0 (0x0)   execution time : 332.849 s
Press any key to continue.
```

Conclusion:

This project shows how data structures like Trie and Linked List can be applied in practical problems. The Trie enables fast dictionary word validation, while the linked list manages scores dynamically. File handling adds flexibility by letting the game use an external dictionary. The game is simple but demonstrates important programming concepts in a fun and interactive way.

References:

- [1] T. H. Cormen, C. Leiserson, R. Rivest, and C. Stein, *Introduction to Algorithms*, MIT Press, 2009.
- [2] B. Kernighan and D. Ritchie, *The C Programming Language*, Prentice Hall, 1988.
- [3] GeeksforGeeks, "Trie Data Structure," [Online]. Available: <https://www.geeksforgeeks.org/trie-insert-and-search/>
- [4] GitHub, *Project Repository Hosting Platform*. Available: <https://github.com>